Contents

OOP Python Cheat Sheet **1**
Tabla de Contenidos . . . 1
1. Conceptos Fundamentales . . . 1
2. Clases y Objetos . . . 2
3. Atributos . . . 3
4. Métodos . . . 4
5. Propiedades . . . 5
6. Herencia . . . 7
7. Clases Abstractas e Interfaces . . . 9
8. Métodos Especiales (Dunder) . . . 11
9. Dataclasses . . . 15
10. Patrones de Diseño . . . 17
Recursos Relacionados . . . 20

OOP Python Cheat Sheet

Python 3.13+ | Programación Orientada a Objetos

Tabla de Contenidos

1. Conceptos Fundamentales
2. Clases y Objetos
3. Atributos
4. Métodos
5. Propiedades
6. Herencia
7. Clases Abstractas e Interfaces
8. Métodos Especiales (Dunder)
9. Dataclasses
10. Patrones de Diseño

---

1. Conceptos Fundamentales

Los 4 Pilares de OOP

Pilar	Descripción	Python
Encapsulamiento	Ocultar datos internos	`_protected`, `__private`
Abstracción	Simplificar complejidad	Clases abstractas, ABC
Herencia	Reutilizar código	`class Child(Parent)`
Polimorfismo	Misma interfaz, diferentes implementaciones	Duck typing, métodos

Terminología

Término	Descripción
Clase	Plantilla/blueprint para crear objetos
Objeto/Instancia	Entidad creada a partir de una clase
Atributo	Variable perteneciente a una clase/objeto
Método	Función definida dentro de una clase
Constructor	`__init__` - inicializa el objeto

Término	Descripción
self	Referencia a la instancia actual
cls	Referencia a la clase actual

---

2. Clases y Objetos

Definición Básica

```python
class Dog:
    """Clase que representa un perro."""

    # Atributo de clase (compartido por todas las instancias)
    species = "Canis familiaris"

    def __init__(self, name: str, age: int) -> None:
        """Constructor - inicializa la instancia."""
        # Atributos de instancia (únicos por objeto)
        self.name = name
        self.age = age

    def bark(self) -> str:
        """Método de instancia."""
        return f"{self.name} says woof!"

    def __str__(self) -> str:
        """Representación legible."""
        return f"Dog({self.name}, {self.age})"

    def __repr__(self) -> str:
        """Representación para debugging."""
        return f"Dog(name={self.name!r}, age={self.age})"
```

Crear y Usar Objetos

```python
# Crear instancias
buddy = Dog("Buddy", 3)
max = Dog("Max", 5)

# Acceder atributos
print(buddy.name)          # "Buddy"
print(buddy.species)       # "Canis familiaris"

# Llamar métodos
print(buddy.bark())        # "Buddy says woof!"

# Modificar atributos
buddy.age = 4

# Verificar tipo
isinstance(buddy, Dog)     # True
type(buddy)                # <class '__main__.Dog'>
```

Type Hints en Clases

```python
from typing import ClassVar, Self

class User:
    # ClassVar indica atributo de clase
    default_role: ClassVar[str] = "user"
    instance_count: ClassVar[int] = 0

    def __init__(self, name: str, email: str) -> None:
        self.name: str = name
        self.email: str = email
        User.instance_count += 1

    def with_role(self, role: str) -> Self:  # Self = tipo de la instancia
        """Retorna una copia con nuevo rol."""
        new_user = User(self.name, self.email)
        new_user.role = role
        return new_user
```

---

3. Atributos

Tipos de Atributos

```python
class Example:
    # Atributo de clase
    class_attr = "shared"

    def __init__(self):
        # Atributo de instancia
        self.instance_attr = "unique"

        # Atributo "protegido" (convención)
        self._protected = "internal use"

        # Atributo "privado" (name mangling)
        self.__private = "hidden"

obj = Example()

# Acceso
obj.class_attr            # "shared"
obj.instance_attr         # "unique"
obj._protected            # "internal use" (accesible, pero no recomendado)
obj.__private             #  AttributeError
obj._Example__private     # "hidden" (name mangling)
```

Atributos de Clase vs Instancia

```python
class Counter:
    # Atributo de clase - compartido
    count = 0

    def __init__(self):
        Counter.count += 1
```

```python
        # Atributo de instancia - único
        self.id = Counter.count

c1 = Counter()
c2 = Counter()

print(Counter.count)      # 2 (compartido)
print(c1.count)           # 2
print(c1.id)              # 1 (único)
print(c2.id)              # 2 (único)

#  Cuidado con mutables en clase
class Bug:
    items = []  #  Compartido entre instancias!

class Correct:
    def __init__(self):
        self.items = []  #  Único por instancia
```

slots (Optimización de Memoria)

```python
class OptimizedUser:
    __slots__ = ['name', 'email', 'age']

    def __init__(self, name: str, email: str, age: int):
        self.name = name
        self.email = email
        self.age = age

user = OptimizedUser("Alice", "alice@example.com", 30)
user.phone = "123"        #  AttributeError - no permitido

# Beneficios:
# - Menor uso de memoria
# - Acceso más rápido
# - Previene atributos dinámicos

# Limitaciones:
# - No __dict__
# - Herencia requiere cuidado
```

---

4. Métodos

Tipos de Métodos

```python
class MyClass:
    class_var = "class level"

    def __init__(self, value: int):
        self.value = value

    # Método de instancia
    def instance_method(self) -> str:
        """Accede a self (instancia)."""
```

```python
        return f"Instance: {self.value}"

    # Método de clase
    @classmethod
    def class_method(cls) -> str:
        """Accede a cls (clase)."""
        return f"Class: {cls.class_var}"

    @classmethod
    def create_default(cls) -> "MyClass":
        """Factory method común."""
        return cls(0)

    # Método estático
    @staticmethod
    def static_method(x: int, y: int) -> int:
        """No accede a self ni cls."""
        return x + y
```

Uso de Métodos

```python
obj = MyClass(42)

# Método de instancia
obj.instance_method()          # "Instance: 42"

# Método de clase
MyClass.class_method()         # "Class: class level"
obj.class_method()             # También funciona

# Método estático
MyClass.static_method(1, 2)    # 3
obj.static_method(1, 2)        # También funciona

# Factory method
default = MyClass.create_default()
```

Cuándo Usar Cada Tipo

Tipo	Usa `self`	Usa `cls`	Caso de Uso
Instancia			Operar con datos del objeto
Clase			Factory methods, acceder/modificar estado de clase
Estático			Utilidades relacionadas pero independientes

---

5. Propiedades

Property Básica

```python
class Circle:
    def __init__(self, radius: float):
        self._radius = radius
```

```python
    @property
    def radius(self) -> float:
        """Getter - acceso de lectura."""
        return self._radius

    @radius.setter
    def radius(self, value: float) -> None:
        """Setter - acceso de escritura."""
        if value < 0:
            raise ValueError("Radius cannot be negative")
        self._radius = value

    @radius.deleter
    def radius(self) -> None:
        """Deleter - eliminar atributo."""
        del self._radius

    @property
    def area(self) -> float:
        """Propiedad calculada (solo lectura)."""
        return 3.14159 * self._radius ** 2

circle = Circle(5)
print(circle.radius)      # 5 (getter)
circle.radius = 10        # setter
print(circle.area)        # 314.159 (calculado)
# circle.area = 100       #  AttributeError - no setter
```

Property sin Setter (Solo Lectura)

```python
class User:
    def __init__(self, first: str, last: str):
        self._first = first
        self._last = last

    @property
    def full_name(self) -> str:
        """Propiedad calculada de solo lectura."""
        return f"{self._first} {self._last}"

user = User("John", "Doe")
print(user.full_name)     # "John Doe"
# user.full_name = "Jane"  #  AttributeError
```

Property con Validación

```python
class Temperature:
    def __init__(self, celsius: float = 0):
        self.celsius = celsius  # Usa el setter

    @property
    def celsius(self) -> float:
        return self._celsius

    @celsius.setter
```

```python
    def celsius(self, value: float) -> None:
        if value < -273.15:
            raise ValueError("Temperature below absolute zero!")
        self._celsius = value

    @property
    def fahrenheit(self) -> float:
        return self._celsius * 9/5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value: float) -> None:
        self.celsius = (value - 32) * 5/9

temp = Temperature(25)
print(temp.fahrenheit)    # 77.0
temp.fahrenheit = 100
print(temp.celsius)       # 37.78
```

---

6. Herencia

Herencia Simple

```python
class Animal:
    def __init__(self, name: str):
        self.name = name

    def speak(self) -> str:
        return "Some sound"

    def info(self) -> str:
        return f"{self.name} is an animal"

class Dog(Animal):
    def __init__(self, name: str, breed: str):
        super().__init__(name)  # Llamar constructor padre
        self.breed = breed

    def speak(self) -> str:  # Override
        return f"{self.name} says woof!"

    def fetch(self) -> str:  # Método nuevo
        return f"{self.name} fetches the ball"

class Cat(Animal):
    def speak(self) -> str:
        return f"{self.name} says meow!"

# Uso
dog = Dog("Buddy", "Labrador")
cat = Cat("Whiskers")

dog.speak()               # "Buddy says woof!"
cat.speak()               # "Whiskers says meow!"
dog.info()                # "Buddy is an animal" (heredado)
```

```
dog.fetch()                # "Buddy fetches the ball"

isinstance(dog, Dog)       # True
isinstance(dog, Animal)    # True
issubclass(Dog, Animal)    # True
```

Herencia Múltiple

```
class Flyable:
    def fly(self) -> str:
        return "Flying!"

class Swimmable:
    def swim(self) -> str:
        return "Swimming!"

class Duck(Animal, Flyable, Swimmable):
    def speak(self) -> str:
        return f"{self.name} says quack!"

duck = Duck("Donald")
duck.fly()                 # "Flying!"
duck.swim()                # "Swimming!"
duck.speak()               # "Donald says quack!"

# MRO (Method Resolution Order)
Duck.__mro__
# (<class 'Duck'>, <class 'Animal'>, <class 'Flyable'>,
#  <class 'Swimmable'>, <class 'object'>)
```

super() y MRO

```
class A:
    def greet(self):
        print("A")

class B(A):
    def greet(self):
        print("B")
        super().greet()

class C(A):
    def greet(self):
        print("C")
        super().greet()

class D(B, C):
    def greet(self):
        print("D")
        super().greet()

d = D()
d.greet()
# D
# B
```

```
# C
# A

# MRO: D -> B -> C -> A -> object
print(D.__mro__)
```

Mixins

```
class LoggerMixin:
    """Mixin que agrega capacidad de logging."""

    def log(self, message: str) -> None:
        print(f"[{self.__class__.__name__}] {message}")

class SerializableMixin:
    """Mixin que agrega serialización a JSON."""

    def to_dict(self) -> dict:
        return {k: v for k, v in self.__dict__.items()
                if not k.startswith('_')}

    def to_json(self) -> str:
        import json
        return json.dumps(self.to_dict())

class User(LoggerMixin, SerializableMixin):
    def __init__(self, name: str, email: str):
        self.name = name
        self.email = email

    def save(self) -> None:
        self.log(f"Saving user {self.name}")
        # Lógica de guardado

user = User("Alice", "alice@example.com")
user.log("Created")       # "[User] Created"
user.to_json()            # '{"name": "Alice", "email": "alice@example.com"}'
```

---

7. Clases Abstractas e Interfaces

ABC (Abstract Base Class)

```
from abc import ABC, abstractmethod

class Shape(ABC):
    """Clase abstracta - no se puede instanciar."""

    @abstractmethod
    def area(self) -> float:
        """Método abstracto - debe ser implementado."""
        pass

    @abstractmethod
    def perimeter(self) -> float:
```

```python
        pass

    def describe(self) -> str:
        """Método concreto - puede ser heredado."""
        return f"Area: {self.area()}, Perimeter: {self.perimeter()}"

# shape = Shape()  #  TypeError - no se puede instanciar

class Rectangle(Shape):
    def __init__(self, width: float, height: float):
        self.width = width
        self.height = height

    def area(self) -> float:
        return self.width * self.height

    def perimeter(self) -> float:
        return 2 * (self.width + self.height)

class Circle(Shape):
    def __init__(self, radius: float):
        self.radius = radius

    def area(self) -> float:
        return 3.14159 * self.radius ** 2

    def perimeter(self) -> float:
        return 2 * 3.14159 * self.radius

rect = Rectangle(5, 3)
rect.area()               # 15
rect.describe()           # "Area: 15, Perimeter: 16"
```

Propiedades Abstractas

```python
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @property
    @abstractmethod
    def max_speed(self) -> int:
        pass

    @abstractmethod
    def start(self) -> str:
        pass

class Car(Vehicle):
    @property
    def max_speed(self) -> int:
        return 200

    def start(self) -> str:
        return "Car engine starting..."
```

Protocol (Structural Subtyping)

```python
from typing import Protocol

class Drawable(Protocol):
    """Interface mediante Protocol - duck typing estático."""

    def draw(self) -> None:
        ...

class Circle:
    def draw(self) -> None:
        print("Drawing circle")

class Square:
    def draw(self) -> None:
        print("Drawing square")

def render(shape: Drawable) -> None:
    """Acepta cualquier objeto con método draw()."""
    shape.draw()

# No necesita heredar de Drawable
render(Circle())          #  Funciona
render(Square())          #  Funciona
```

---

8. Métodos Especiales (Dunder)

Inicialización y Representación

```python
class Point:
    def __init__(self, x: float, y: float):
        """Constructor - inicializa la instancia."""
        self.x = x
        self.y = y

    def __repr__(self) -> str:
        """Representación para desarrolladores (debug)."""
        return f"Point(x={self.x}, y={self.y})"

    def __str__(self) -> str:
        """Representación para usuarios."""
        return f"({self.x}, {self.y})"

    def __del__(self):
        """Destructor - llamado cuando el objeto es eliminado."""
        print(f"Point {self} destroyed")

p = Point(3, 4)
repr(p)                   # "Point(x=3, y=4)"
str(p)                    # "(3, 4)"
print(p)                  # "(3, 4)"
```

Comparación

```python
from functools import total_ordering

@total_ordering  # Genera los demás métodos a partir de __eq__ y __lt__
class Version:
    def __init__(self, major: int, minor: int, patch: int):
        self.major = major
        self.minor = minor
        self.patch = patch

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Version):
            return NotImplemented
        return (self.major, self.minor, self.patch) == \
               (other.major, other.minor, other.patch)

    def __lt__(self, other: "Version") -> bool:
        if not isinstance(other, Version):
            return NotImplemented
        return (self.major, self.minor, self.patch) < \
               (other.major, other.minor, other.patch)

    def __hash__(self) -> int:
        return hash((self.major, self.minor, self.patch))

v1 = Version(1, 0, 0)
v2 = Version(2, 0, 0)
v1 < v2                   # True
v1 <= v2                  # True (generado por @total_ordering)
v1 == Version(1, 0, 0)    # True
```

Operadores Aritméticos

```python
class Vector:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    def __add__(self, other: "Vector") -> "Vector":
        """self + other"""
        return Vector(self.x + other.x, self.y + other.y)

    def __sub__(self, other: "Vector") -> "Vector":
        """self - other"""
        return Vector(self.x - other.x, self.y - other.y)

    def __mul__(self, scalar: float) -> "Vector":
        """self * scalar"""
        return Vector(self.x * scalar, self.y * scalar)

    def __rmul__(self, scalar: float) -> "Vector":
        """scalar * self"""
        return self * scalar

    def __neg__(self) -> "Vector":
```

```python
        """-self"""
        return Vector(-self.x, -self.y)

    def __abs__(self) -> float:
        """abs(self) - magnitud"""
        return (self.x**2 + self.y**2) ** 0.5

    def __repr__(self) -> str:
        return f"Vector({self.x}, {self.y})"

v1 = Vector(3, 4)
v2 = Vector(1, 2)

v1 + v2                   # Vector(4, 6)
v1 - v2                   # Vector(2, 2)
v1 * 2                    # Vector(6, 8)
2 * v1                    # Vector(6, 8)
-v1                       # Vector(-3, -4)
abs(v1)                   # 5.0
```

Operadores de Contenedor

```python
class Inventory:
    def __init__(self):
        self._items: dict[str, int] = {}

    def __len__(self) -> int:
        """len(self)"""
        return len(self._items)

    def __contains__(self, item: str) -> bool:
        """item in self"""
        return item in self._items

    def __getitem__(self, item: str) -> int:
        """self[item]"""
        return self._items[item]

    def __setitem__(self, item: str, quantity: int) -> None:
        """self[item] = quantity"""
        self._items[item] = quantity

    def __delitem__(self, item: str) -> None:
        """del self[item]"""
        del self._items[item]

    def __iter__(self):
        """for item in self"""
        return iter(self._items)

    def __reversed__(self):
        """reversed(self)"""
        return reversed(list(self._items))

inv = Inventory()
inv["sword"] = 1          # __setitem__
```

```python
inv["potion"] = 5
len(inv)                    # 2 - __len__
"sword" in inv              # True - __contains__
inv["sword"]                # 1 - __getitem__
del inv["sword"]            # __delitem__
for item in inv:            # __iter__
    print(item)
```

Context Manager

```python
class FileManager:
    def __init__(self, filename: str, mode: str):
        self.filename = filename
        self.mode = mode
        self.file = None

    def __enter__(self):
        """Llamado al entrar en 'with'."""
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        """Llamado al salir de 'with'."""
        if self.file:
            self.file.close()
        # Retornar True suprime la excepción
        return False

with FileManager("test.txt", "w") as f:
    f.write("Hello!")
# Archivo se cierra automáticamente
```

Callable

```python
class Multiplier:
    def __init__(self, factor: int):
        self.factor = factor

    def __call__(self, value: int) -> int:
        """Permite llamar al objeto como función."""
        return value * self.factor

double = Multiplier(2)
triple = Multiplier(3)

double(5)                   # 10
triple(5)                   # 15
callable(double)            # True
```

Lista Completa de Dunders

Categoría	Métodos
Inicialización	`__init__`, `__new__`, `__del__`
Representación	`__repr__`, `__str__`, `__format__`, `__bytes__`

Categoría	Métodos
Comparación	`__eq__`, `__ne__`, `__lt__`, `__le__`, `__gt__`, `__ge__`
Hash	`__hash__`, `__bool__`
Atributos	`__getattr__`, `__setattr__`, `__delattr__`, `__getattribute__`
Contenedor	`__len__`, `__getitem__`, `__setitem__`, `__delitem__`, `__contains__`, `__iter__`, `__next__`, `__reversed__`
Numéricos	`__add__`, `__sub__`, `__mul__`, `__truediv__`, `__floordiv__`, `__mod__`, `__pow__`, `__neg__`, `__abs__`
Context	`__enter__`, `__exit__`
Callable	`__call__`

---

9. Dataclasses

Básico

```python
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

# Equivale a escribir:
# - __init__
# - __repr__
# - __eq__

p1 = Point(3, 4)
p2 = Point(3, 4)
print(p1)                # Point(x=3, y=4)
p1 == p2                 # True
```

Con Opciones

```python
from dataclasses import dataclass, field

@dataclass(
    frozen=True,         # Inmutable
    order=True,          # Genera __lt__, __le__, etc.
    slots=True,          # Usa __slots__
)
class User:
    name: str
    email: str
    age: int = 0         # Valor por defecto

    # Campo con factory por defecto
    tags: list[str] = field(default_factory=list)

    # Campo excluido de comparación
```

```
    internal_id: int = field(default=0, compare=False)

    # Campo excluido de __repr__
    password: str = field(default="", repr=False)

user = User("Alice", "alice@example.com", age=30)
# user.name = "Bob"       #  FrozenInstanceError (frozen=True)
```

Post-init y Campos Calculados

```
from dataclasses import dataclass, field

@dataclass
class Rectangle:
    width: float
    height: float
    area: float = field(init=False)  # No incluir en __init__

    def __post_init__(self):
        """Llamado después de __init__."""
        self.area = self.width * self.height

rect = Rectangle(5, 3)
rect.area                 # 15
```

Herencia en Dataclasses

```
from dataclasses import dataclass

@dataclass
class Person:
    name: str
    age: int

@dataclass
class Employee(Person):
    employee_id: str
    department: str

emp = Employee("Alice", 30, "E001", "Engineering")
```

Conversión y Serialización

```
from dataclasses import dataclass, asdict, astuple

@dataclass
class Config:
    host: str
    port: int
    debug: bool = False

config = Config("localhost", 8080)

# A diccionario
asdict(config)            # {'host': 'localhost', 'port': 8080, 'debug': False}
```

```python
# A tupla
astuple(config)           # ('localhost', 8080, False)

# A JSON
import json
json.dumps(asdict(config))
```

---

10. Patrones de Diseño

Singleton

```python
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

s1 = Singleton()
s2 = Singleton()
s1 is s2                  # True

# Alternativa con decorador
def singleton(cls):
    instances = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance

@singleton
class Database:
    pass
```

Factory Method

```python
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def speak(self) -> str:
        pass

class Dog(Animal):
    def speak(self) -> str:
        return "Woof!"

class Cat(Animal):
    def speak(self) -> str:
        return "Meow!"

class AnimalFactory:
```

```python
    @staticmethod
    def create(animal_type: str) -> Animal:
        match animal_type.lower():
            case "dog":
                return Dog()
            case "cat":
                return Cat()
            case _:
                raise ValueError(f"Unknown animal: {animal_type}")

# Uso
dog = AnimalFactory.create("dog")
dog.speak()               # "Woof!"
```

Builder

```python
from dataclasses import dataclass

@dataclass
class Pizza:
    size: str
    cheese: bool
    pepperoni: bool
    mushrooms: bool
    olives: bool

class PizzaBuilder:
    def __init__(self):
        self._size = "medium"
        self._cheese = False
        self._pepperoni = False
        self._mushrooms = False
        self._olives = False

    def size(self, size: str) -> "PizzaBuilder":
        self._size = size
        return self

    def add_cheese(self) -> "PizzaBuilder":
        self._cheese = True
        return self

    def add_pepperoni(self) -> "PizzaBuilder":
        self._pepperoni = True
        return self

    def add_mushrooms(self) -> "PizzaBuilder":
        self._mushrooms = True
        return self

    def add_olives(self) -> "PizzaBuilder":
        self._olives = True
        return self

    def build(self) -> Pizza:
        return Pizza(
```

```
            self._size, self._cheese, self._pepperoni,
            self._mushrooms, self._olives
        )

# Uso con chaining
pizza = (PizzaBuilder()
    .size("large")
    .add_cheese()
    .add_pepperoni()
    .build())
```

Observer

```
from abc import ABC, abstractmethod

class Observer(ABC):
    @abstractmethod
    def update(self, message: str) -> None:
        pass

class Subject:
    def __init__(self):
        self._observers: list[Observer] = []

    def attach(self, observer: Observer) -> None:
        self._observers.append(observer)

    def detach(self, observer: Observer) -> None:
        self._observers.remove(observer)

    def notify(self, message: str) -> None:
        for observer in self._observers:
            observer.update(message)

class EmailNotifier(Observer):
    def update(self, message: str) -> None:
        print(f"Email: {message}")

class SMSNotifier(Observer):
    def update(self, message: str) -> None:
        print(f"SMS: {message}")

# Uso
news = Subject()
news.attach(EmailNotifier())
news.attach(SMSNotifier())
news.notify("Breaking news!")
# Email: Breaking news!
# SMS: Breaking news!
```

Dependency Injection

```
from abc import ABC, abstractmethod

class Database(ABC):
```

```python
    @abstractmethod
    def query(self, sql: str) -> list:
        pass

class PostgreSQL(Database):
    def query(self, sql: str) -> list:
        return [f"PostgreSQL: {sql}"]

class MySQL(Database):
    def query(self, sql: str) -> list:
        return [f"MySQL: {sql}"]

class UserRepository:
    def __init__(self, db: Database):  # Inyección por constructor
        self._db = db

    def get_all(self) -> list:
        return self._db.query("SELECT * FROM users")

# Uso - fácil de intercambiar implementaciones
repo_pg = UserRepository(PostgreSQL())
repo_mysql = UserRepository(MySQL())

# Testing - inyectar mock
class MockDatabase(Database):
    def query(self, sql: str) -> list:
        return [{"id": 1, "name": "Test User"}]

repo_test = UserRepository(MockDatabase())
```

Recursos Relacionados

- **Anterior**: Data Structures Cheat Sheet
- **Siguiente**: File Handling Cheat Sheet
- **PEP 8**: Style Guide for Python Code
- **PEP 557**: Data Classes

Cheat Sheet creado para el Bootcamp Python Zero to Hero - 2026